

// UNREAL ENGINE 5.8 · C++ IN ORB ARENA

UNREAL C++ Cheat-Sheet

Spin it. Hit it. Spawn it. Ship it.



LOG + TYPES

Say it in the log

```
UE_LOG(LogTemp, Warning,
  TEXT("HP=%d %s"), Hp, *Name);
int32  float  FString  FName  FText
```

TEXT() on literals. A * before an FString!

DEBUG

Say it on screen

```
if (GEngine) {
  GEngine->AddOnScreenDebugMessage(
    -1, 2.f, FColor::Yellow,
    TEXT("Ouch!"));
}
```

ACTOR MUST MEMORIZE

Your first Actor

```
UCLASS()
class AOrb : public AActor {
  GENERATED_BODY()
public:
  AOrb();
  void BeginPlay() override;
  void Tick(float Dt) override;
};
```

Call Super::Tick(Dt) first. #include .generated.h LAST.

UPROPERTY + UFUNCTION

Knobs for the editor

```
UPROPERTY(EditAnywhere,
  BlueprintReadWrite,
  Category="Orb")
float Spin = 90.f;
UFUNCTION(BlueprintCallable)
void Collect();
```

COMPONENTS

Actors are built from parts

```
UPROPERTY(VisibleAnywhere)
TObjectPtr<UStaticMeshComponent>
  Mesh;
// in the constructor:
Mesh = CreateDefaultSubobject<
  UStaticMeshComponent>(
  TEXT("Mesh"));
RootComponent = Mesh;
Light->SetupAttachment(Mesh);
```

FVECTOR

Move and spin

```
FVector P = GetActorLocation();
P.Z += 50.f;
SetActorLocation(P);
AddActorLocalRotation(
  FRotator(0.f, Spin * Dt, 0.f));
```

X forward, Y right, Z up.

OVERLAP + CAST

Something touched me

```
Sphere->OnComponentBeginOverlap
  .AddDynamic(this, &AOrb::OnHit);
UFUNCTION()   // required!
void OnHit(UPrimitiveComponent* C,
  AActor* Other,
  UPrimitiveComponent* OC,
  int32 Idx, bool bSweep,
  const FHitResult& Hit);
if (ACharacter* P =
    Cast<ACharacter>(Other)) { }
```

TARRAY + TMAP

Unreal's containers

```
TArray<int32> Scores;
Scores.Add(10);
Scores.Num();        // size
for (int32 S : Scores) { }
TMap<FString, int32> Pts;
Pts.Add(TEXT("gold"), 50);
```

JS: array and Map

UENUM + USTRUCT

Typed data for your orbs

```
UENUM(BlueprintType)
enum class EOrbKind : uint8 {
  Gold, Ruby
};
USTRUCT(BlueprintType)
struct FOrbData {
  GENERATED_BODY()
  UPROPERTY(EditAnywhere)
  int32 Points = 10;
};
```

GAME MODE

The referee

```
UCLASS()
class AOrbGM : public AGameModeBase {
  GENERATED_BODY()
};
AOrbGM* GM = GetWorld()
  ->GetAuthGameMode<AOrbGM>();
```

SPAWN + TIMER

Things appear over time

```
UPROPERTY(EditAnywhere)
TSubclassOf<AOrb> OrbClass;
GetWorld()->SpawnActor<AOrb>(
  OrbClass, Loc,
  FRotator::ZeroRotator);
FTimerHandle T;
GetWorldTimerManager().SetTimer(
  T, this, &AGM::Spawn, 2.f, true);
```

ENEMY MOVEMENT

Chase the player

```
#include "Kismet/GameplayStatics.h"
APawn* P = UGameplayStatics::
  GetPlayerPawn(this, 0);
FVector Dir = (P->GetActorLocation()
  - GetActorLocation())
  .GetSafeNormal();
AddMovementInput(Dir, 1.f);
```

Needs a Pawn or Character.

GARBAGE COLLECTION MUST MEMORIZE

Memory the Unreal way

```
UPROPERTY()
AActor* Keep;    // GC-safe
TWeakObjectPtr<AActor> Maybe;
if (Maybe.IsValid()) { }
if (IsValid(Ptr)) { }
Destroy();   // remove an Actor
```

A raw UObject pointer without UPROPERTY() can dangle.

DELEGATES

Your own events

```
DECLARE_DYNAMIC_MULTICAST_DELEGATE_OneParam(
  FOnScore, int32, NewScore);
UPROPERTY(BlueprintAssignable)
FOnScore OnScore;
OnScore.Broadcast(Score);
GM->OnScore.AddDynamic(
  this, &AHud::Show);
```

JS: addEventListener

BLUEPRINT BRIDGE

C++ and Blueprints

```
UFUNCTION(BlueprintCallable)
  // Blueprint calls your C++
UFUNCTION(
  BlueprintImplementableEvent)
  // C++ calls a Blueprint
UFUNCTION(BlueprintNativeEvent)
  // C++ default, BP overrides
```

NAME TAGS

The first letter tells you what it is

A	Actor	AOrb
U	Object / Component	UStaticMeshComponent
F	struct / value type	FVector
E	enum	EOrbKind
T	template / container	TArray
I	interface	IInteractable
b	bool variable	bAlive



Boss fights: common bugs



Log shows garbage FString in UE_LOG needs a * in front.

Editor ignores your handler AddDynamic functions need UFUNCTION().

Crash on a dead Actor Keep UObject pointers in UPROPERTY() or use TWeakObjectPtr.

Include order #include "Orb.generated.h" goes LAST in the header.

Frozen Tick Start with Super::Tick(DeltaTime).

Spawn, don't new Make Actors with SpawnActor, never with new.

Null crash Check IsValid(P) before you use P→Foo().

Semicolon A class or struct ends with }; (the semicolon!)